LAPORAN PRAKTIKUM

ALGORITMA PEMROGRAMAN 2

MODUL 3 & 4 – FUNGSI DAN PROSEDUR

NAMA : AIS NURHANIFAH

NIM : 109082500162

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Dalam pemrograman menggunakan bahasa Go, fungsi dan prosedur merupakan blok kode yang dirancang untuk melakukan tugas spesifik secara modular. Fungsi didefinisikan sebagai sub-program yang menerima input melalui parameter dan mengembalikan suatu nilai (*return value*) ke pemanggilnya. Sementara itu, prosedur dalam konteks Go sebenarnya adalah fungsi yang tidak mengembalikan nilai, melainkan langsung melakukan aksi seperti mencetak output ke layar. Penggunaan keduanya sangat krusial untuk menerapkan prinsip *Don't Repeat Yourself* (DRY), sehingga kode menjadi lebih terstruktur, mudah dibaca, dan efisien karena logika yang sama tidak perlu ditulis berulang kali.

Selain struktur dasar, pemahaman mengenai mekanisme pengiriman argumen sangat penting, yaitu antara *pass-by-value* dan *pass-by-reference*. Pada *pass-by-value*, program membuat salinan dari nilai variabel sehingga perubahan di dalam fungsi tidak memengaruhi variabel asli. Sebaliknya, *pass-by-reference* menggunakan pointer untuk mengirimkan alamat memori variabel asli, sehingga perubahan yang terjadi di dalam fungsi akan bersifat permanen pada variabel tersebut. Selain itu, bahasa Go mendukung konsep komposisi fungsi dan fungsi rekursif atau iteratif kompleks, seperti pada implementasi perhitungan faktorial atau deret bilangan, yang memungkinkan penyelesaian masalah matematika yang rumit secara sistematis.

Struktur kontrol seperti percabangan if-else dan switch-case, serta perulangan for, sering kali diintegrasikan di dalam fungsi untuk menciptakan logika program yang dinamis. switch-case sangat efektif digunakan dalam fungsi yang memiliki banyak pilihan kondisi, seperti pada program kalkulator bangun datar, karena membuat alur eksekusi lebih rapi dibandingkan *nested-if*. Integrasi antara fungsi modular dengan struktur kontrol ini memungkinkan pengembang untuk membangun aplikasi yang interaktif, di mana input dari pengguna diproses melalui validasi dan kalkulasi yang terisolasi di dalam blok fungsi tertentu.

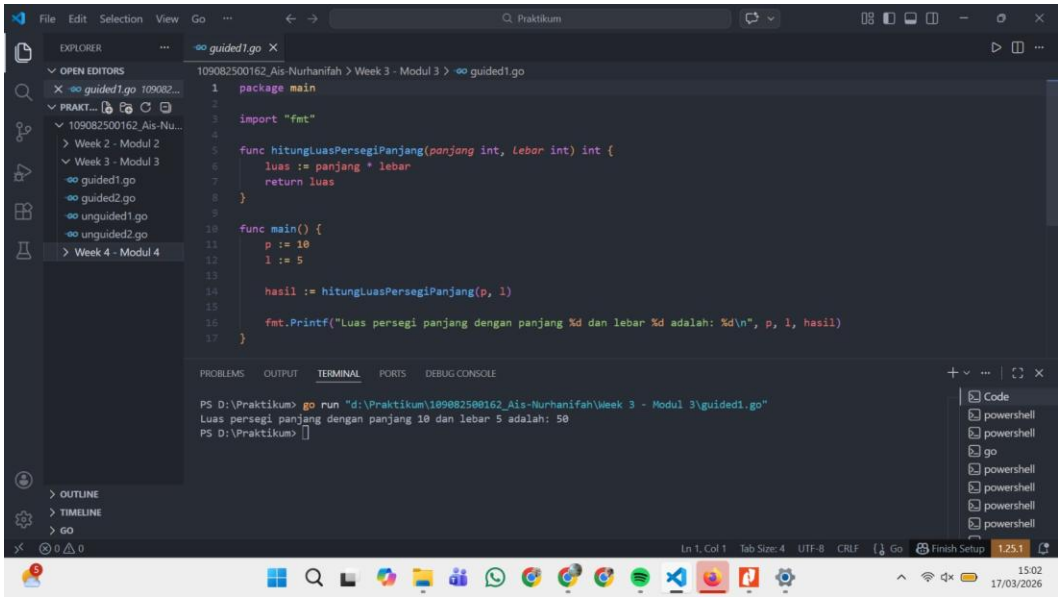
B. Guided

1. [luasPersegiPanjang.go]

```
package main

import "fmt"

func hitungLuasPersegiPanjang(panjang int, lebar int)
int {
	luas := panjang * lebar
	return luas
}

func main() {
```

```
        p := 10
        l := 5

        hasil := hitungLuasPersegiPanjang(p, l)

        fmt.Printf("Luas persegi panjang dengan panjang
    %d dan lebar %d adalah: %d\n", p, l, hasil)
}
```

Output :



Penjelasan :

Program di atas merupakan implementasi bahasa pemrograman Go yang dirancang untuk menghitung luas sebuah persegi panjang dengan memanfaatkan konsep fungsi modular. Struktur program dimulai dengan deklarasi fungsi hitungLuasPersegiPanjang yang memiliki dua parameter input, yaitu panjang dan lebar, keduanya bertipe data integer. Di dalam fungsi tersebut, terdapat operasi aritmetika perkalian untuk menghitung luas, yang hasilnya kemudian disimpan ke dalam variabel lokal luas sebelum dikembalikan ke pemanggil fungsi melalui instruksi return. Pendekatan ini menunjukkan prinsip enkapsulasi logika perhitungan agar kode lebih rapi dan dapat digunakan kembali secara efisien.

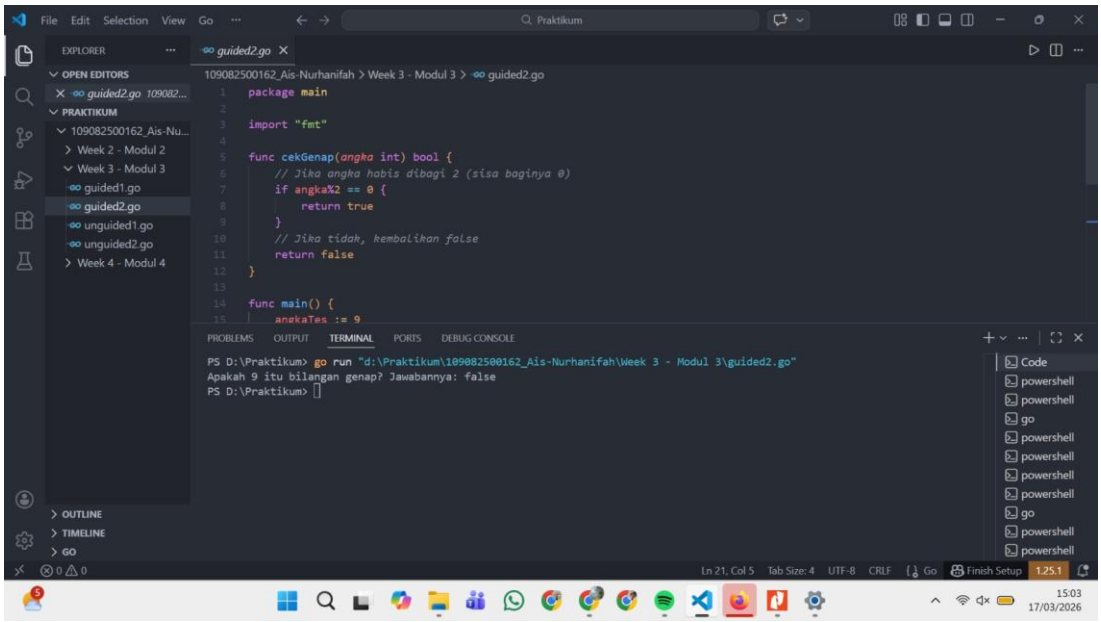
Pada bagian func main(), program menentukan nilai dimensi persegi panjang secara statis dengan menginisialisasi variabel p bernilai 10 dan l bernilai 5. Selanjutnya, program melakukan pemanggilan fungsi hitungLuasPersegiPanjang dengan mengirimkan kedua variabel tersebut sebagai argumen, dan hasilnya ditampung dalam variabel hasil. Untuk menampilkan output ke layar, digunakan

fungsi fmt.Printf dengan format specifier %d untuk mencetak nilai bilangan bulat secara berurutan. Berdasarkan eksekusi pada terminal, program berhasil menghitung hasil perkalian 10 dan 5, kemudian menampilkan kalimat informatif yang menyatakan bahwa luas persegi panjang tersebut adalah 50.

2. [cekGenap.go]

```
package main

import "fmt"

func cekGenap(angka int) bool {
	// Jika angka habis dibagi 2 (sisa baginya 0)
	if angka%2 == 0 {
		return true
	}
	// Jika tidak, kembalikan false
	return false
}

func main() {
	angkaTes := 9

	// Memanggil fungsi dan menyimpan hasil kembaliannya
	hasilGenap := cekGenap(angkaTes)

	fmt.Printf("Apakah %d itu bilangan genap? Jawabannya: 
%t\n", angkaTes, hasilGenap)
}
```

Output :



Penjelasan :

Program di atas merupakan implementasi sederhana dalam bahasa pemrograman Go yang bertujuan untuk menentukan apakah suatu bilangan bulat termasuk kategori bilangan genap atau ganjil menggunakan fungsi dengan nilai kembalian (*return value*). Pada bagian awal, program mendefinisikan sebuah fungsi bernama cekGenap yang menerima satu parameter input bertipe integer (angka) dan mengembalikan nilai boolean (true atau false). Logika utama di dalam fungsi ini memanfaatkan operator **modulo** (%), di mana ekspresi angka % 2 == 0 digunakan untuk memeriksa sisa hasil bagi. Jika sebuah angka dibagi dua menghasilkan sisa nol, maka fungsi akan mengembalikan nilai true yang menandakan bilangan tersebut genap, dan sebaliknya akan mengembalikan false jika terdapat sisa bagi.

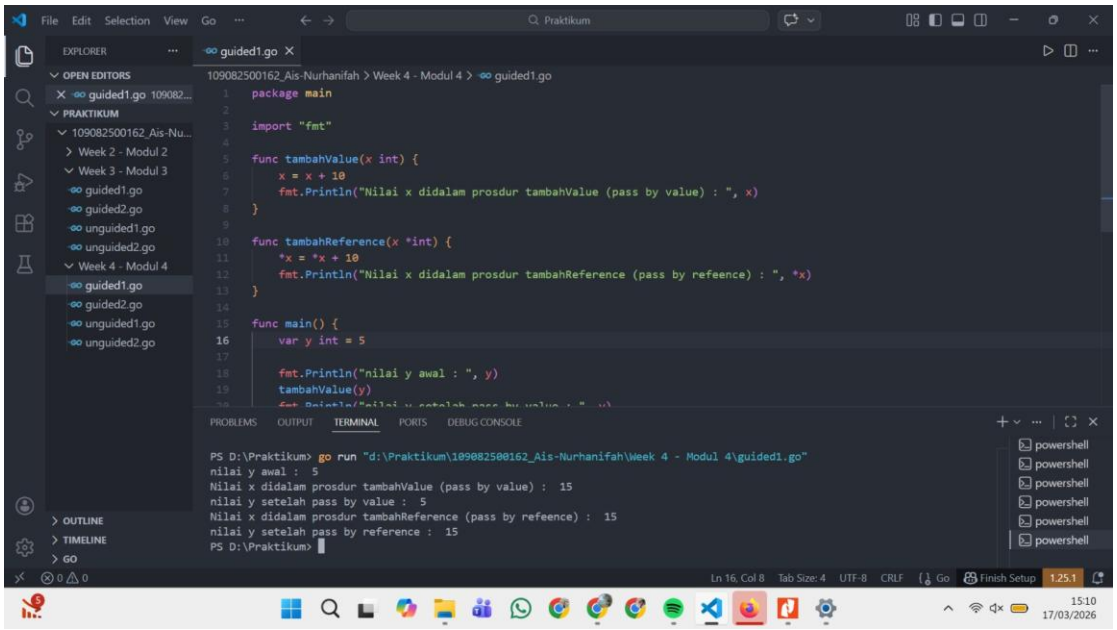
Pada fungsi utama atau func main(), program melakukan inisialisasi variabel angkaTes dengan nilai 9. Variabel ini kemudian dikirim sebagai argumen ke dalam fungsi cekGenap. Hasil evaluasi dari fungsi tersebut disimpan ke dalam variabel baru bernama hasilGenap. Untuk menampilkan hasilnya ke terminal, program menggunakan fmt.Printf dengan bantuan *verb* %d untuk mencetak angka desimal dan %t untuk mencetak nilai boolean secara literal. Berdasarkan output yang terlihat pada terminal di gambar, karena angka 9 menghasilkan sisa bagi 1 ketika dibagi 2, maka program secara akurat menampilkan jawaban false yang menunjukkan bahwa 9 bukanlah bilangan genap.

3. [valueReference.go]

```
package main

import "fmt"

func tambahValue(x int) {
      x = x + 10
      fmt.Println("Nilai x didalam prosdur tambahValue (pass
by value) : ", x)
}

func tambahReference(x *int) {
      *x = *x + 10
      fmt.Println("Nilai x didalam prosdur tambahReference
(pass by refeence) : ", *x)
}
```

```
func main() {
      var y int = 5

      fmt.Println("nilai y awal : ", y)
      tambahValue(y)
      fmt.Println("nilai y setelah pass by value : ", y)

      tambahReference(&y)
      fmt.Println("nilai y setelah pass by reference : ", y)
}
```

Output :



Penjelasan :

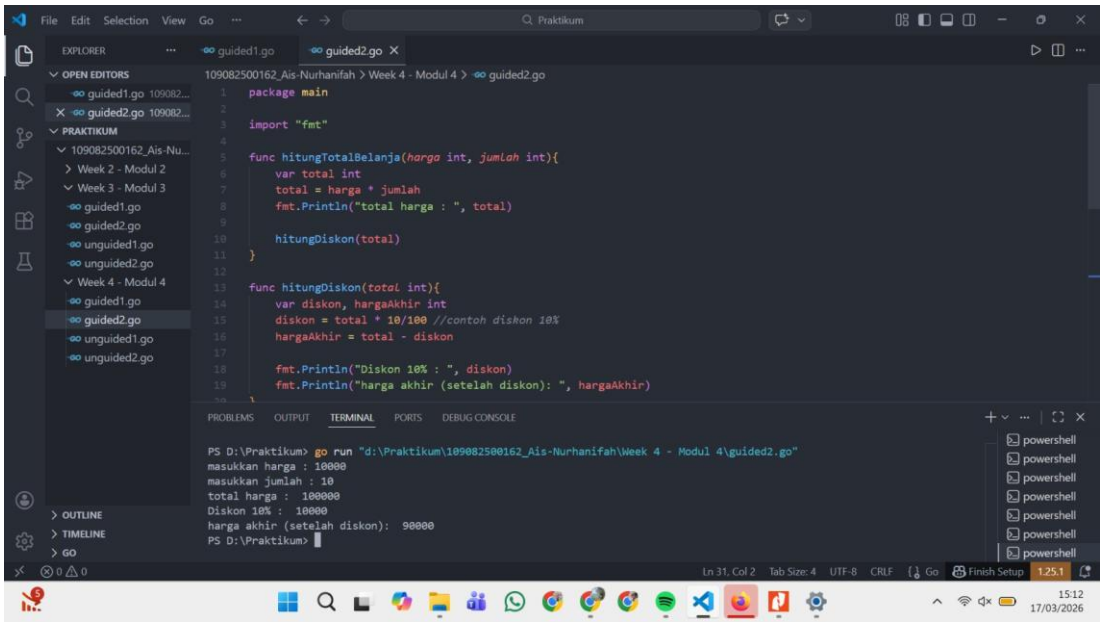
Program ini dirancang untuk mendemonstrasikan perbedaan mendasar antara mekanisme pengiriman argumen secara ***pass-by-value*** dan ***pass-by-reference*** dalam bahasa pemrograman Go. Pada fungsi tambahValue, program menggunakan *pass-by-value*, di mana nilai dari variabel y disalin ke parameter x. Perubahan yang terjadi di dalam fungsi tersebut (menambah nilai sebesar 10) hanya berlaku pada salinan variabel di dalam lingkup (*scope*) fungsi itu sendiri. Akibatnya, sebagaimana terlihat pada output terminal, nilai variabel y di fungsi main tetap bernilai 5 meskipun fungsi tambahValue telah dijalankan, karena variabel asli di memori tidak tersentuh oleh operasi di dalam fungsi tersebut.

Sebaliknya, fungsi tambahReference menggunakan mekanisme *pass-by-reference* dengan bantuan **pointer** (*int). Saat fungsi ini dipanggil menggunakan operator *address-of* (&y), yang dikirimkan bukanlah nilainya, melainkan alamat memori dari variabel y. Di dalam fungsi, operator dereferensi (*x) digunakan untuk mengakses dan mengubah nilai langsung pada alamat memori tersebut. Hal ini menyebabkan perubahan bersifat permanen dan memengaruhi variabel asli. Output program membuktikan hal ini, di mana nilai y setelah pemanggilan tambahReference berubah menjadi 15. Program ini memberikan pemahaman penting bahwa penggunaan pointer sangat krusial ketika kita ingin sebuah fungsi memodifikasi data asli atau untuk efisiensi memori saat menangani data berukuran besar.

4. [Diskon.go]

```
package main

import "fmt"

func hitungTotalBelanja(harga int, jumlah int){
      var total int
      total = harga * jumlah
      fmt.Println("total harga : ", total)

      hitungDiskon(total)
}

func hitungDiskon(total int){
      var diskon, hargaAkhir int
      diskon = total * 10/100 //contoh diskon 10%
      hargaAkhir = total - diskon

      fmt.Println("Diskon 10% : ", diskon)
      fmt.Println("harga akhir (setelah diskon): ",
hargaAkhir)
}

func main(){
      var harga, jumlah int

      fmt.Print("masukkan harga : ")
      fmt.Scan(&harga)
      fmt.Print("masukkan jumlah : ")
      fmt.Scan(&jumlah)

      hitungTotalBelanja(harga, jumlah)
}
```

Output :



Penjelasan :

Program ini merupakan simulasi sederhana dari sistem kasir yang dirancang untuk menghitung total belanja dan potongan harga (diskon) secara otomatis. Struktur program ini menarik karena menerapkan konsep **pemanggilan fungsi berantai** (*function chaining*). Alur dimulai pada fungsi main, di mana program menerima input interaktif dari pengguna berupa harga barang dan jumlah barang menggunakan fmt.Scan. Data tersebut kemudian diteruskan ke fungsi hitungTotalBelanja untuk dikalikan sehingga didapatkan nilai total kotor. Setelah nilai total dihitung dan ditampilkan, fungsi ini tidak langsung selesai, melainkan memanggil fungsi lain yaitu hitungDiskon dengan membawa nilai total tersebut sebagai argumen.

Pada fungsi hitungDiskon, program melakukan kalkulasi potongan harga sebesar 10% dengan formula total * 10 / 100. Setelah nilai diskon didapatkan, program menghitung harga akhir yang harus dibayar oleh pelanggan dengan mengurangkan total awal dengan nilai diskon tersebut. Semua hasil perhitungan, mulai dari total harga, nominal diskon, hingga harga akhir, ditampilkan ke terminal menggunakan fmt.Println. Berdasarkan contoh eksekusi pada gambar, jika pengguna memasukkan harga 10.000 dengan jumlah 10, program secara akurat menghitung total 100.000, memberikan diskon 10.000, dan menghasilkan harga akhir sebesar 90.000.

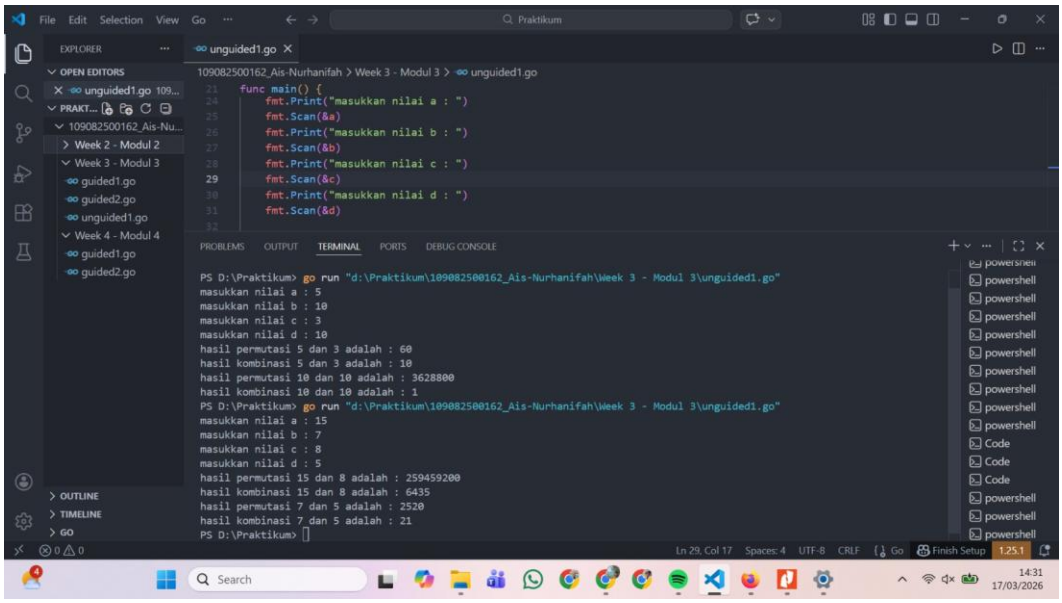
C. Unguided

1. Soal Unguided 1

Jawaban :

```
package main

import "fmt"

func factorial(n int) int {
    hasil := 1
    for i := 1; i <= n; i++ {
        hasil = hasil * i
    }
    return hasil
}

func permutasi(n, r int) int {
    return factorial(n) / factorial(n-r)
}

func kombinasi(n, r int) int {
    return factorial(n) / (factorial(r) * factorial(n-
r))
}

func main() {
    var a, b, c, d int

    fmt.Print("masukkan nilai a : ")
    fmt.Scan(&a)
    fmt.Print("masukkan nilai b : ")
    fmt.Scan(&b)
    fmt.Print("masukkan nilai c : ")
    fmt.Scan(&c)
    fmt.Print("masukkan nilai d : ")
    fmt.Scan(&d)

    fmt.Printf("hasil permutasi %d dan %d adalah :
%d\n", a, c, permutasi(a, c))

    fmt.Printf("hasil kombinasi %d dan %d adalah :
%d\n", a, c, kombinasi(a, c))

    fmt.Printf("hasil permutasi %d dan %d adalah :
%d\n", b, d, permutasi(b, d))

    fmt.Printf("hasil kombinasi %d dan %d adalah :
%d\n", b, d, kombinasi(b, d))
}
```

Output :



Penjelasan :

Program ini dirancang dengan pendekatan modular yang membagi logika perhitungan menjadi beberapa fungsi spesifik, yaitu factorial, permutasi, dan kombinasi. Fungsi factorial bekerja sebagai mesin penghitung dasar menggunakan perulangan for untuk mengalikan deret bilangan, yang kemudian dipanggil oleh fungsi permutasi dan kombinasi untuk mengimplementasikan rumus matematika sesuai kaidah peluang. Di dalam fungsi main, program secara interaktif mengambil empat input variabel dari pengguna menggunakan fmt.Scan dengan operator *address-of* (&) untuk menyimpan nilai langsung ke alamat memori. Struktur ini memastikan efisiensi kode karena logika perhitungan faktorial yang kompleks hanya ditulis satu kali namun dapat digunakan berulang kali untuk berbagai keperluan rumus yang berbeda.

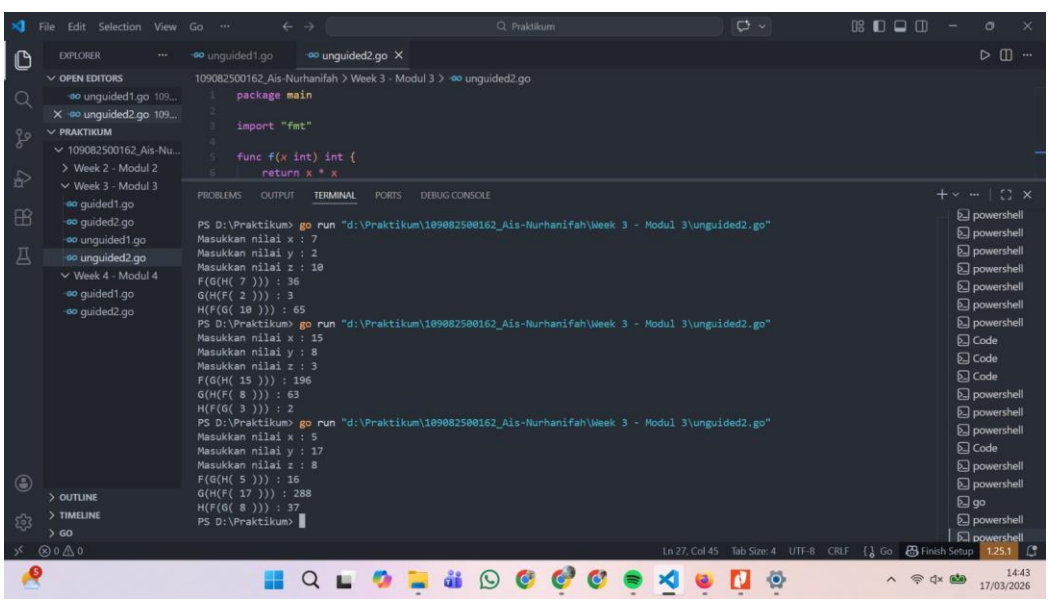
Bagian output pada terminal menunjukkan keberhasilan program dalam memproses data secara *real-time* dan menyajikannya dalam format kalimat informatif menggunakan fmt.Printf. Melalui hasil eksekusi, terlihat perbedaan signifikan antara hasil permutasi dan kombinasi; misalnya pada pasangan angka 10 dan 10, nilai permutasi mencapai 3.628.800 karena memperhitungkan urutan susunan, sementara nilai kombinasinya adalah 1 karena hanya ada satu cara memilih seluruh objek tanpa memedulikan urutannya. Akurasi angka yang muncul membuktikan bahwa alur logika dari input ke fungsi faktorial hingga ke tampilan layar telah berjalan selaras dengan teori matematika diskrit. Output yang sistematis ini memudahkan pengguna untuk memverifikasi hasil perhitungan secara cepat dan akurat.

2. Soal Unguided 2

Jawaban :

```go
package main

import "fmt"

func f(x int) int {
    return x * x
}
func g(x int) int {
    return x - 2
}
func h(x int) int {
    return x + 1
}
func main() {
    var x, y, z int

    fmt.Print("Masukkan nilai x : ")
    fmt.Scan(&x)
    fmt.Print("Masukkan nilai y : ")
    fmt.Scan(&y)
    fmt.Print("Masukkan nilai z : ")
    fmt.Scan(&z)

    fmt.Printf("F(G(H( %d ))) : %d\n", x, f(g(h(x))))

    fmt.Printf("G(H(F( %d ))) : %d\n", y, g(h(f(y))))

    fmt.Printf("H(F(G( %d ))) : %d\n", z, h(f(g(z))))
}
```

Output :



Penjelasan :

Program ini dimulai dengan pendeklarasian package main dan impor pustaka fmt untuk menangani fungsi input dan output. Inti dari program ini adalah pembuatan tiga fungsi matematika dasar, yaitu f(x) untuk menghitung kuadrat (x^2), g(x) untuk pengurangan (x - 2), dan h(x) untuk penambahan (x + 1). Ketiga fungsi ini dirancang secara modular agar dapat dipanggil secara bersarang, yang dalam matematika dikenal dengan konsep komposisi fungsi. Di dalam fungsi utama atau func main(), program menyiapkan tiga variabel integer (x, y, z) dan menggunakan perintah fmt.Scan disertai operator *pointer* (&) untuk mengambil input nilai langsung dari pengguna melalui terminal.

Pada bagian akhir program, dilakukan proses kalkulasi komposisi fungsi yang kompleks dan hasilnya ditampilkan menggunakan fmt.Printf. Alur eksekusi fungsi dilakukan dari dalam ke luar; sebagai contoh, pada ekspresi f(g(h(x))), program akan menjalankan fungsi h terlebih dahulu, hasilnya dimasukkan ke fungsi g, dan hasil akhirnya diproses oleh fungsi f. Berdasarkan output di terminal, jika pengguna memasukkan nilai x=7, maka urutannya adalah (7+1) = 8, lalu (8-2) = 6, dan terakhir 6^2 = 36. Pendekatan ini menunjukkan bagaimana bahasa Go dapat menyelesaikan persoalan matematika berjenjang dengan kode yang tetap bersih dan terstruktur, serta memudahkan praktikan dalam memverifikasi hasil perhitungan manual dengan hasil eksekusi program secara langsung.

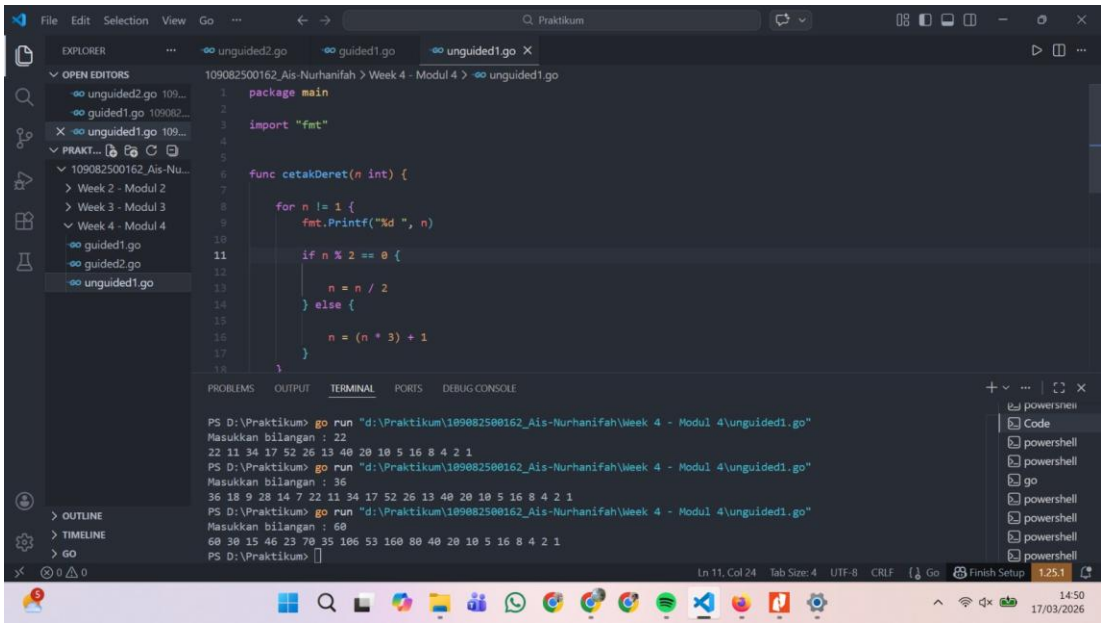
3. Soal Unguided 3

Jawaban :

```
package main
package main

import "fmt"

func cetakDeret(n int) {

    for n != 1 {
        fmt.Printf("%d ", n)

        if n % 2 == 0 {

            n = n / 2
        } else {

            n = (n * 3) + 1
        }
```

```
        }

        fmt.Println(1)
    }

    func main() {
        var bilangan int

        fmt.Print("Masukkan bilangan : ")
        fmt.Scan(&bilangan)

        cetakDeret(bilangan)
    }
```

Output :



Penjelasan :

Program ini merupakan implementasi algoritma iteratif untuk menghasilkan deret bilangan berdasarkan aturan matematika tertentu yang bergantung pada sifat genap atau ganjilnya suatu angka. Struktur program dimulai dengan fungsi cetakDeret yang menerima satu parameter input berupa bilangan bulat n. Di dalam fungsi ini, terdapat struktur perulangan for yang akan terus berjalan selama nilai n belum mencapai angka 1. Logika di dalam perulangan menggunakan percabangan if-else untuk mengevaluasi nilai n; jika bilangan tersebut genap (n % 2 == 0), maka nilainya akan dibagi dua, namun jika ganjil, nilainya akan dikalikan tiga dan ditambah satu (n = (n * 3) + 1).

Pada bagian func main(), program berfungsi sebagai antarmuka interaktif yang meminta pengguna memasukkan satu bilangan bulat melalui perintah fmt.Scan. Bilangan tersebut kemudian dikirimkan ke fungsi cetakDeret untuk diproses dan ditampilkan hasilnya ke terminal secara horizontal. Berdasarkan hasil eksekusi pada gambar, terlihat bahwa program mampu menangani berbagai input seperti angka 22, 36, dan 60, di mana masing-masing menghasilkan deret angka yang berbeda namun selalu berakhir pada angka 1. Hal ini menunjukkan efektivitas penggunaan logika kontrol alur (*control flow*) dalam menyelesaikan permasalahan algoritma yang bersifat dinamis.

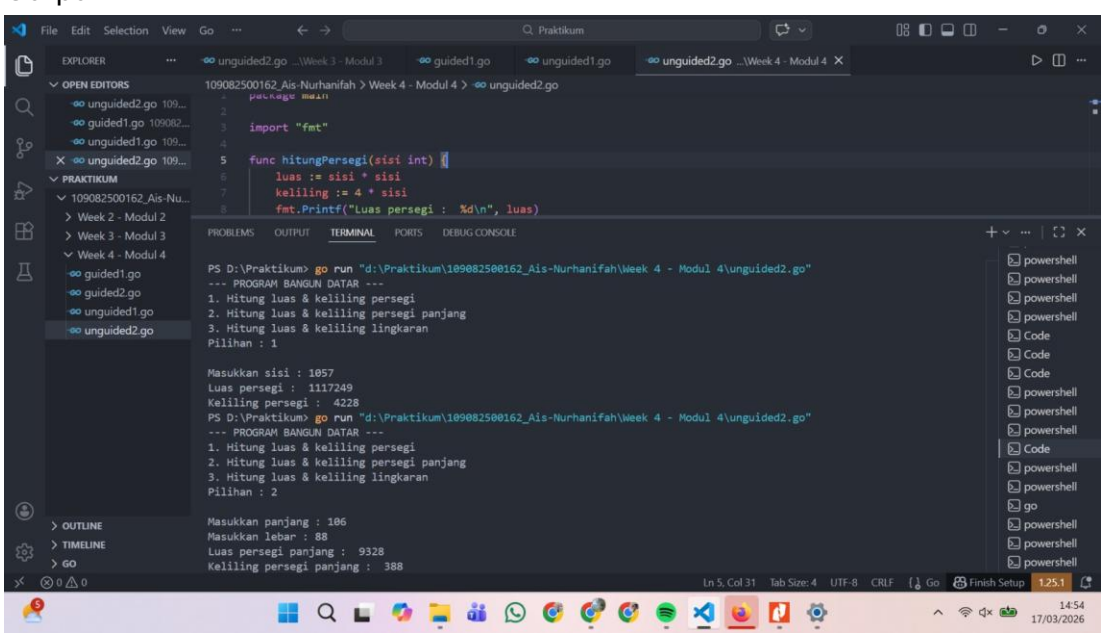
4. Soal Unguided 4

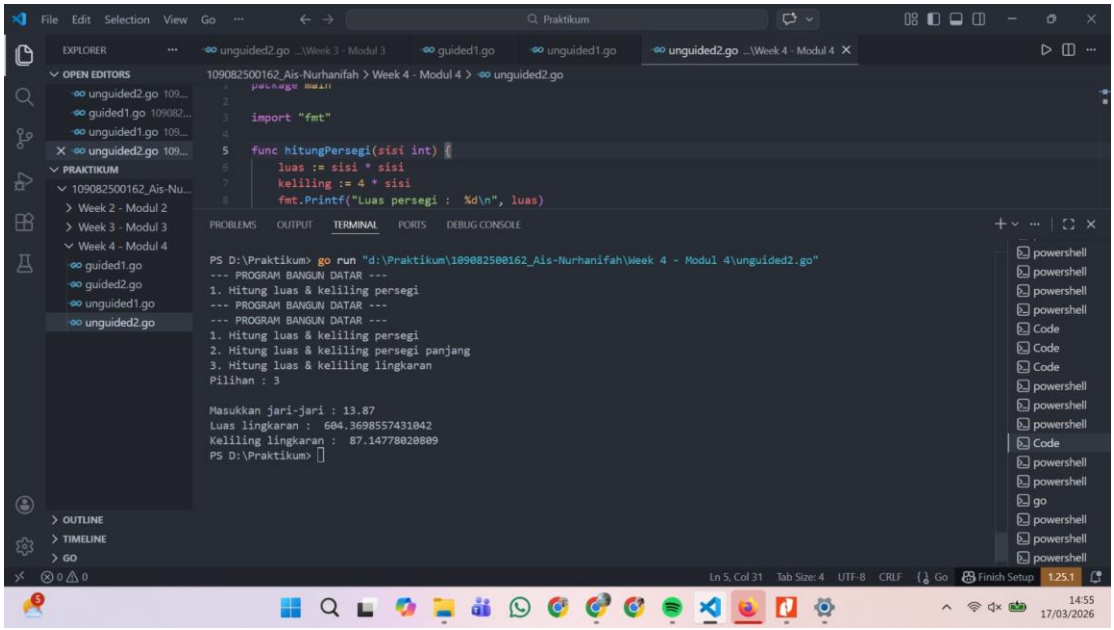
   Jawaban :

```
package main

import "fmt"

func hitungPersegi(sisi int) {
    luas := sisi * sisi
    keliling := 4 * sisi
    fmt.Printf("Luas persegi :  %d\n", luas)
    fmt.Printf("Keliling persegi :  %d\n", keliling)
}

func hitungPersegiPanjang(p, l int) {
    luas := p * l
    keliling := 2 * (p + l)
    fmt.Printf("Luas persegi panjang :  %d\n", luas)
    fmt.Printf("Keliling persegi panjang :  %d\n",
keliling)
}

func hitungLingkaran(r float64) {
    luas := 3.1415926535 * r * r
    keliling := 2 * 3.1415926535 * r
    fmt.Printf("Luas lingkaran :  %g\n", luas)
    fmt.Printf("Keliling lingkaran :  %g\n", keliling)
}

func main() {
    var pilihan int

    fmt.Println("--- PROGRAM BANGUN DATAR ---")
    fmt.Println("1. Hitung luas & keliling persegi")
    fmt.Println("2. Hitung luas & keliling persegi
panjang")
    fmt.Println("3. Hitung luas & keliling lingkaran")
    fmt.Print("Pilihan : ")
```

```go
        fmt.Scan(&pilihan)
        fmt.Println()

        switch pilihan {
        case 1:
            var s int
            fmt.Print("Masukkan sisi : ")
            fmt.Scan(&s)
            hitungPersegi(s)
        case 2:
            var p, l int
            fmt.Print("Masukkan panjang : ")
            fmt.Scan(&p)
            fmt.Print("Masukkan lebar : ")
            fmt.Scan(&l)
            hitungPersegiPanjang(p, l)
        case 3:
            var r float64
            fmt.Print("Masukkan jari-jari : ")
            fmt.Scan(&r)
            hitungLingkaran(r)
        default:
            fmt.Println("Pilihan tidak ada!")
        }
    }
```

Output :





Penjelasan :

Program ini dirancang sebagai aplikasi kalkulator geometri interaktif yang memanfaatkan struktur kontrol **switch-case** dan pembagian tugas melalui fungsi-fungsi modular untuk menghitung luas dan keliling berbagai bangun datar. Di dalamnya terdapat tiga prosedur utama dengan logika aritmetika yang spesifik: hitungPersegi untuk menghitung luas (s x s) dan keliling (4 x s), hitungPersegiPanjang untuk luas (p x l) dan keliling (2 x (p+l)), serta hitungLingkaran yang menggunakan tipe data float64 untuk mengakomodasi nilai presisi tinggi dari konstanta phi guna menghitung luas (phi x r^2) dan keliling (2 x phi x r). Pemisahan logika kalkulasi ini ke dalam fungsi-fungsi terpisah bertujuan untuk meningkatkan keterbacaan kode (*code readability*) dan memudahkan pemeliharaan program, karena setiap bangun datar memiliki unit pemrosesannya sendiri.

Pada fungsi main, program berperan sebagai pengendali alur yang menyajikan menu pilihan kepada pengguna secara sekuensial. Setelah pengguna memasukkan angka pilihan (1, 2, atau 3), blok switch akan mengarahkan alur eksekusi ke case yang sesuai untuk meminta dimensi yang diperlukan—seperti sisi, panjang, lebar, atau jari-jari—lalu memanggil prosedur kalkulasi yang relevan. Jika pengguna memasukkan angka di luar pilihan yang tersedia, blok default akan memberikan pesan peringatan bahwa pilihan tidak ditemukan. Berdasarkan hasil eksekusi di terminal, program terbukti mampu menangani berbagai skenario input, mulai dari angka bulat besar untuk persegi panjang

hingga angka desimal dengan presisi tinggi pada lingkaran, serta menampilkan seluruh hasil kalkulasi secara sistematis dengan format yang mudah dipahami.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa fungsi dan prosedur dalam bahasa Go sangat penting untuk membuat program menjadi lebih terstruktur, rapi, dan mudah dipahami. Dengan menggunakan fungsi, program dapat dibagi menjadi bagian-bagian kecil sesuai tugasnya, sehingga kode tidak perlu ditulis berulang kali dan lebih efisien. Selain itu, praktikum ini juga menunjukkan perbedaan antara pass-by-value dan pass-by-reference. Pass-by-value hanya menyalin nilai sehingga tidak mengubah data asli, sedangkan pass-by-reference menggunakan pointer sehingga perubahan di dalam fungsi akan memengaruhi variabel asli. Pemahaman ini penting untuk mengontrol bagaimana data diproses di dalam program.

Melalui berbagai contoh seperti perhitungan luas, pengecekan bilangan, permutasi, kombinasi, hingga program interaktif bangun datar, terlihat bahwa fungsi dapat digunakan untuk menyelesaikan berbagai masalah secara sistematis. Penggunaan struktur kontrol seperti if-else, switch-case, dan perulangan juga membantu membuat alur program menjadi lebih dinamis.

Secara keseluruhan, praktikum ini membantu memahami dasar penting dalam pemrograman, yaitu bagaimana menyusun logika program dengan baik menggunakan fungsi, prosedur, dan pengelolaan data yang tepat agar program menjadi lebih efektif dan mudah dikembangkan.